Module 4 Group Assignment - Architecture Design

Member Participation
Ulises Almonte – 100%
Thong Ho – 100%
Chetana Kamraj Patel – 100%
Suvarna Ramu – 100%

Mason Online Course Registration System Re-Architecture

I. What would be an appropriate architecture pattern / style for this system? Why

Recommended Architecture Pattern: Layered Architecture with Client-Server Components

Primary Pattern Selection

For Mason's online course registration system, a Layered Architecture is recommended as the primary architectural pattern, enhanced with Client-Server components for distributed access. This hybrid approach addresses the system's complexity and operational requirements.

Rationale for Layered Architecture

The layered architecture pattern is particularly well-suited for this application for several reasons:

Portability and Platform Independence: The students and administrators can access the system from diverse devices and platforms (laptops, tablets, mobile devices running various operating systems). Hence, a layered architecture will support portability. The separation of concerns across layers ensures that changes to one platform do not require redesigning the entire system.

Modifiability and Maintainability: The registration system requires frequent updates, new courses are added, policies change, and academic calendars expand. The layered architecture emphasis on modifiability allows development teams to update business logic without affecting the user interface or database layers. This modularity is critical for a system that must adapt to changing university requirements each semester.

Reusability Across Academic Units: Different schools and departments within Mason (e.g., Volgenau School of Engineering, College of Science, School of Business) share common registration workflows but may have unique requirements. A layered architecture enables reusable components in the core business logic layer while allowing customization at higher layers for department-specific needs.

Security Through Layer Isolation: The layered pattern supports robust security by placing critical assets in the inner layers. Student data, course enrollment records, and payment information can be protected in secure inner layers, with authentication and authorization mechanisms controlling access at appropriate boundaries.

Integration with Client-Server Pattern

The **Client-Server Architecture** complements the layered approach by addressing the distributed nature of the registration system:

Scalability for Peak Load: During registration periods, thousands of students simultaneously access the system. The client-server model enables horizontal scaling by deploying multiple server instances behind load balancers, distributing the processing load across infrastructure resources.

Centralized Data Management: Course availability, seat counts, and prerequisite checking require consistent, centralized data management. A server-side repository ensures all clients receive accurate, real-time information and prevents race conditions when multiple students attempt to register for limited-seat courses.

Browser-Based Accessibility: Following the example of the iLearn system architecture, a browser-based user interface layer allows students to access registration from any internet connected device without installing specialized software, maximizing accessibility and convenience.

II. What are some non-functional requirements/quality attributes that must be achieved for such a system?

To ensure the success of the re-architected Mason registration system, the following Quality Attributes must be achieved. These represent the system’s operational constraints and are critical to maintaining stakeholder trust during high-stakes registration periods:

Availability: The system must maintain high availability, particularly during peak registration windows. To balance this with Maintainability, the architecture formally defines a weekly two-hour maintenance window on weekends. This allows for proactive serviceability such as security patching and schema updates without infringing upon the system's operational requirements during high-stakes enrollment periods.

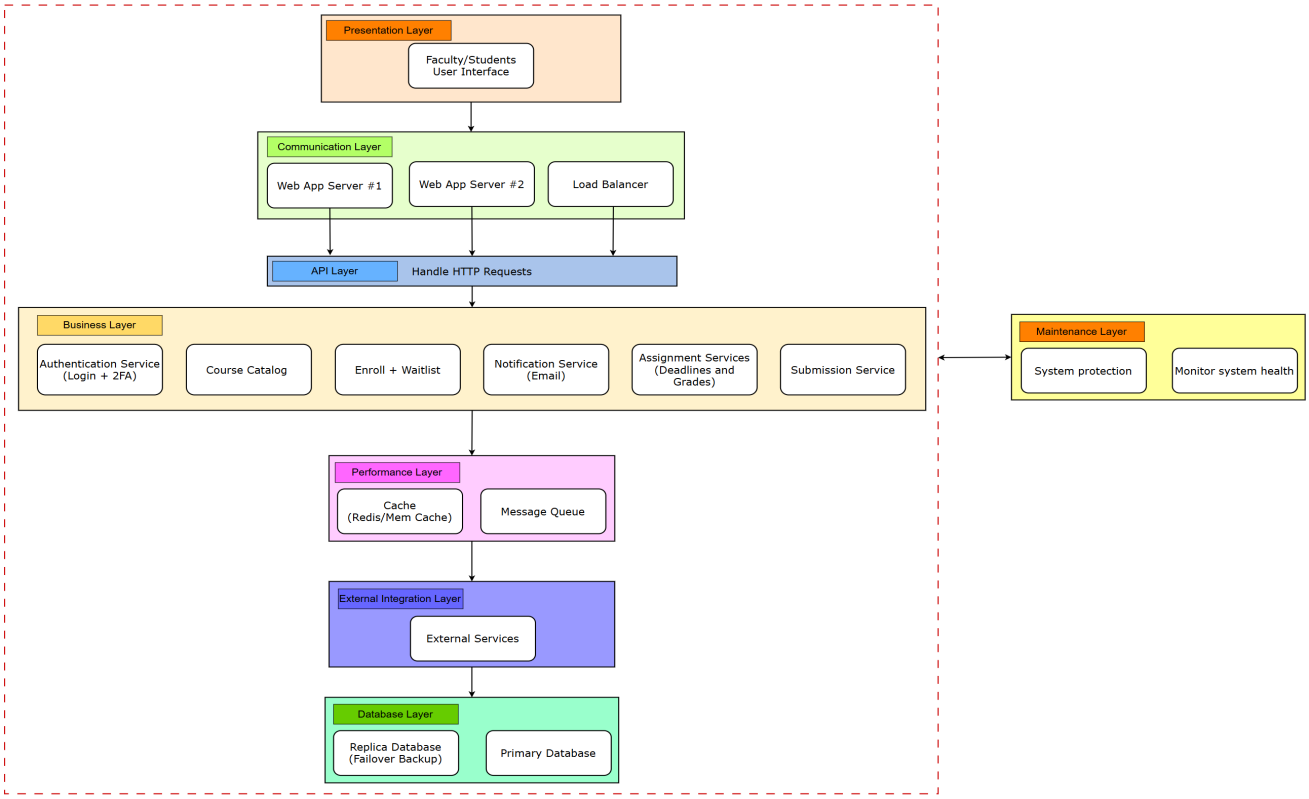
Security and Data Integrity: As the system handles sensitive personal information, it must adhere to lawful data safety standards (e.g., FERPA). By implementing robust Authentication and Authorization within the system utilities layer as seen in the iLearn system architecture, we ensure that only verified users can access records and prevent the unauthorized manipulation of waitlists or course data.

Usability and Interoperability: In alignment with ADA standards, the system must be accessible to students with disabilities. Furthermore, the design must prioritize Interoperability, ensuring the registration portal remains fully functional across multiple different browsers and is viewable on various screen sizes.

Scalability and Reliability: The architecture must handle extreme traffic spikes without failure. By utilizing a Service Oriented approach, the system will run multiple application instances to redirect and balance traffic. This redundancy provides Fault Tolerance, ensuring that if one instance fails, the registration process remains reliable for the student.

III. Visual Representation of the Architecture

Below is a visual architectural diagram for Mason's online course registration system, illustrating the layered architecture with client-server components:



Mason Online Course Registration System – Architecture Explanation

1. Presentation Layer (User Access)

- Students and faculty interact with the system through web interfaces.
- Multiple web application servers handle user requests.
- A load balancer distributes incoming traffic across servers.
- This prevents crashes during peak registration times.

2. Communication Layer

- Load balancer: Efficiently distributes incoming user traffic across multiple web servers to prevent bottlenecks and ensure system availability.
- Web app server: A scalable cluster of servers that processes user requests and manages session state, ensuring availability.

3. Application / API Layer (Request Routing)

- All requests pass through an API Gateway.
- The gateway routes request the correct backend service.
- It also helps enforce security and request validation.
- This layer acts as a traffic controller between users and system services.

4. Business Logic Layer (Core Services)

Independent services handle major system functions:

- **Authentication Service** – login and two-factor verification
- **Course Catalog Service** – view available courses.
- **Enrollment & Waitlist Service** – register, drop, manage seat limits.
- **Notification Service** – email confirmations and alerts

These services are separated so they can scale or update independently.

5. Processing & Performance Layer

- **Message Queue**
    - Handles background tasks.
    - Prevents request loss if a service temporarily fails.
- **Cache (Redis/Memcache)**
    - Stores frequently accessed data such as course listings.
    - Reduces database load.
    - Improves response time during heavy usage.

6. External System Integration

The system communicates with other university systems:

- Identity provider (student login verification/Single Sign-On)
- Academic records system: transcript, program, concentration, standing, and degree.

7. Data Layer (Storage)

- The **Primary Database** stores all official registration and academic data.
- The **Replica Database** provides failover support and reads for scalability.
- This configuration ensures data consistency, availability, and fault tolerance.

8. Cross-Cutting Infrastructure (System Protection/Maintenance)

- Monitoring and logging servers track system health.
- Alerts detect failures early.
- Firewall/Web Application Firewall (WAF) protects against security attacks.

Hence, this layered microservices architecture improves scalability, availability, security, and maintainability, allowing the system to handle heavy registration traffic while remaining reliable and adaptable